

claude-windows / 188dd06f-9c99-487c-aa7d-b0bee83c629c

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\188dd06f-9c99-487c-aa7d-b0bee83c629c\subagents\workflows\wf_d9866476-800\agent-a738fee132dfdf9d5.jsonl`
- SHA-256: `e38ee5ec6691776c6f2ff106c095bdb1fb5a3c060d45fd06c02f955eef52d2d2`
- Source modified: `2026-06-22T09:19:48+00:00`
- Imported at: `2026-07-05T16:48:28+00:00`
- project: `wf_d9866476-800`
- session_id: `188dd06f-9c99-487c-aa7d-b0bee83c629c`
```

Transcript

1. user (2026-06-22T09:17:39.672Z)

You are mapping the badminton-highlight-indexer repo, checked out at origin/master under C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/nightly-e2e. Read files under that ABSOLUTE path (Read/Grep/Glob). Be concrete: cite file:line / file::symbol. The goal is to design a NIGHTLY regression that runs the CONFIGURED + CHECKPOINTED pipeline on 2-3 short golden videos and asserts the NUMBER OF REELS (highlight clips) does not change. Return raw structured data, not prose for a human.

SYNTHESIZE the design for the E2E reel-count nightly regression, grounded ONLY in the verified findings below. Produce MARKDOWN with:

- 1) Precise definition of the asserted signal ("number of reels") + how to read it headlessly.
- 2) The recommended HYBRID design: TIER-A = local full-pipeline reel-count regression on 2-3 short owned golden videos (the faithful configured+checkpointed test; GPU or cached); TIER-B = a GitHub-CI deterministic CPU-motion smoke on a tiny fixture (always-on, portable pattern). Be explicit about what each tier CAN and CANNOT test, and WHY the real WASB model can't run in GitHub CI.
- 3) The extensible manifest pattern (adding a video = one row + freeze its expected count).
- 4) Determinism handling + exact-vs-tolerance assertion.
- 5) Reuse of the PR #202 pattern (baseline JSON, dated report, exit codes, scheduled-task wrapper).
- 6) A short "OPEN DECISIONS FOR OWNER" list (2-4 crisp forks) with a recommended default for each.
- 7) A concrete file/implementation plan (new files, edits, tests, CI cron, scheduling).

MAPPING FINDINGS:

```
{
  "reel-path": {
    "summary": "\nMapped the reel-production path end-to-end in the badminton-highlight-indexer.
The regression test signal is: **COUNT OF PLAY_SEGMENTS ROWS in the SQLite database per video
after segmentation + filtering**. One stitched output reel (ONE .mp4 file per /api/compile call)
is created from N filtered play_segments. \n\nThe PIPELINE FLOW:\n1. /api/process (or CLI
--video-path) ? _execute_processing ? video_id = MD5(video_file)\n2. Segmenter.process_video ?
detects rally intervals ? inserts rows into play_segments table\n3. /api/compile receives
segment_ids ? queries play_segments ? applies stitching.min_shot_count filter ? calls
VideoCompiler.compile_highlights with (start_time, end_time) pairs\n4. VideoCompiler trims each
segment (with begin/end padding), re-encodes with optional watermark, ffmpeg concat demuxer
stitches N trimmed clips ? ONE .mp4 reel output file\n\n**Regression signal:** `SELECT COUNT(*)
FROM play_segments WHERE video_id = ?` after segmentation completes. This is stable,
deterministic, and queryable headlessly via the database.\n",
    "key_facts": [
      "video_id = compute_video_id(local_path) = MD5 hash
(backend/utils/hashing.py:compute_video_id)",
      "play_segments table schema: id (AUTOINCREMENT), video_id, start_time, end_time, duration,
server, receiver, metadata (JSON). Rows inserted by segmenter via db.add_play_segment().",
      "Segmentation path: main.py::_execute_processing (line 439) ? segmenter.process_video() ?
```

```

returns list of rally dicts ? each inserted as play_segments row via
db_instance.add_play_segment()),
    "Compilation path: /api/compile (line 705) ? query_play_segments(video_id, {}) ? filter by
segment_ids + stitching.min_shot_count (from metadata['shot_count']) ? extract time_pairs
[(start, end), ...] ? VideoCompiler.compile_highlights(video_path, time_pairs,
output_filename)",
    "VideoCompiler.compile_highlights (compiler.py line 305): normalizes timestamps ? merges
overlapping ranges ? for each segment: _trim_one_segment() (line 192) ffmpeg -ss -to re-encodes
with padding + watermark ? ffmpeg concat demuxer stitches clips ? ONE output .mp4 file",
    "Stitching config (backend/config/models.py line 255): begin_padding=0.5s, end_padding=0.5s
(per-segment), min_shot_count=1 (metadata filter), watermark enabled by default",
    "Output: ONE .mp4 file per /api/compile call at output_dir/{output_filename}. Stored flat (no
video_id subdirs) ? /api/reels lists all .mp4 files in output_dir.",
    "Regression signal (stable, headless, deterministic): `SELECT COUNT(*) FROM play_segments
WHERE video_id = <video_id>` after _execute_processing completes (after destroy-AFTER-confirm
finalization at main.py line 536).",
    "Database path: global_config.output.output_dir / global_config.output.db_name (default:
output/rally_indexer.db)",
    "CLI single-shot headless mode: `indexer --video-path <path> [--segmenter <name>] [--output-
dir <dir>]` (no web UI, exits after segmentation) ? main.py line 997-1075"
],
    "recommendation": "\nFor the NIGHTLY E2E REGRESSION on 2?3 golden videos:\n\n**REGRESSION
SIGNAL:** Count of play_segments rows in the output DB for each golden video. Expected counts
should be hardcoded from HEAD baseline.\n\n**EXACT HEADLESS COMMANDS:**\n\n``bash\n# 1. Run
segmentation (single-shot CLI mode, no server, no UI):\nindexer --video-path
/path/to/golden_video.mp4 \\\n      --segmenter motion \\\n      --output-dir
output/nightly_e2e\n\n# 2. Query the segment count (read-only DB inspection):\npython -c "\n
from backend.infrastructure.database import Database\n db =
Database('output/nightly_e2e/rally_indexer.db')\n from backend.utils.hashing import
compute_video_id\n video_id = compute_video_id('/path/to/golden_video.mp4')\n with
db._connect() as conn:\n     count = conn.cursor().execute(\n         'SELECT COUNT(*) FROM
play_segments WHERE video_id = ?', \n         (video_id,)\n     ).fetchone()[0]\n
print(count)\n"\n\n# 3. Assert count == baseline for each video:\n#     Fail (exit 1) if ANY
video's count deviates\n\n\n**NIGHTLY REGRESSION HARNESS STRUCTURE:**\n\n1. **Checkpointed
baseline:** `eval_baselines/nightly_e2e_baselines.json` (commit into repo)\n     ``json\n {\n
"golden_videos": [\n     {\n         "name": "video1.mp4", "md5": "<video_id>",
"expected_segment_count": 3},\n     {\n         "name": "video2.mp4", "md5": "<video_id>",
"expected_segment_count": 5},\n     {\n         "name": "video3.mp4", "md5": "<video_id>",
"expected_segment_count": 4}\n     ],\n     "generator": "HEAD <commit>",\n
"timestamp": "2026-06-22T00:00:00Z"\n     }\n     ``\n\n2. **Nightly runner script:** Run
headlessly on each golden video ? count play_segments ? compare to baseline ? exit 1 on
mismatch\n\n3. **STITCHING PIPELINE VERIFICATION (optional, post-segmentation):**\n     - After
segment count passes, optionally call /api/compile via Python to produce a test reel\n     -
Assert output .mp4 file exists and has size > 0\n     - (Stitching is more complex ? depends on
segment timing + ffmpeg + watermark rendering; segment count is the stable proxy)\n\n**EXACT
INTEGER TO TRACK:**\n- **Primary:** `COUNT(*) FROM play_segments WHERE video_id = <video_id>`
after segmentation\n- **Secondary (optional):** COUNT of .mp4 files in output_dir that match the
compile request (should be 1 per /api/compile call)\n\n**CONFIG FOR GOLDEN VIDEOS:**\n- Use
`--segmenter motion` (offline, no API key, no GPU ? reproducible)\n- Default padding/watermark
(stitching.begin_padding=0.5, end_padding=0.5)\n- No cloud storage (use local paths only)\n",
    "confidence": "high",
    "code_refs": [
        "backend/main.py:439::_execute_processing",
        "backend/main.py:559::process_video",
        "backend/main.py:705::compile_highlights",
        "backend/main.py:997-1075",
        "backend/infrastructure/database.py:24::Database",
        "backend/infrastructure/database.py:564::query_play_segments",
        "backend/infrastructure/database.py:459::add_play_segment",
        "backend/pipeline/stitcher/compiler.py:18::VideoCompiler",
        "backend/pipeline/stitcher/compiler.py:305::compile_highlights",
        "backend/config/models.py:255::StitchingConfig",
        "backend/utils/hashing.py::compute_video_id",
        "scripts/nightly_regression.py (pattern for existing tripwire harness structure)"
    ]
],

```

```

"risks_or_unknowns": [
    "play_segments count can change if segmenter sensitivity shifts (motion_threshold, other
detector params) ? baseline must be re-snapshotted when segmenter config changes intentionally",
    "Destroy-AFTER-confirm semantics (prune_segments) mean count is stable AFTER
_finalize_reingest (main.py:536) but intermediate count during segmentation run can fluctuate ?
ensure query AFTER job completes",
    "Golden video paths must be stable (local, checked in or mounted);
compute_video_id(local_path) will differ if the path/content changes",
    "Watermark font rendering depends on OS fontconfig + ffmpeg capabilities ? watermark encode
can fail and retry without it (compiler.py:274-283), but segment count is unaffected (only
affects output reel quality)"
]
},
"determinism": {
    "summary": "REEL COUNT DETERMINISM: YES, the COUNT of rally segments (reels/highlights) IS
DETERMINISTIC and SAFE TO ASSERT EXACTLY for a fixed (video, config, weights) triple. The
pipeline has no RNG, dict-ordering issues, GPU nondeterminism, ffmpeg frame variance, or float
threshold ties affecting the COUNT (though byte/timing stability differs).",
    "key_facts": [
        "WASB inference (wasb_infer.py:497): Uses torch.no_grad() and a deterministic detector pass.
Detector outputs per-frame candidates via ordered sliding windows (no shuffling). The tracker
runs a single deterministic re-pass over CACHED detections sorted by frame_id (line 550:
sorted(det_by_fid.keys())), producing a trajectory CSV with stable row order.",
        "Windowing logic (tracknet_runner.py:281-399): Pure deterministic algorithms with NO RNG,
sorted operations, and stable intermediate structures. Active frames are identified by velocity
thresholds (line 275: velocity > velocity_thresh, strict comparison). Window merging (line 362)
uses <= frame-gap comparisons (no ties). Splitting (lines 431-476) selects the largest gap
deterministically (line 461: if gap > best_gap). Midpoint selection (line 435: min() with key
function) is deterministic for any given group. No dict/set iterations, no random seeds.",
        "Trajectory-to-windows conversion (trajectory_to_action_windows): All operations on sorted
lists. Groups are formed sequentially via frame ordering and gap detection, NEVER via dict/set
iteration. Split logic (lines 454-476) uses an iterative worklist popped from a list (not a
set), maintaining order. Final window filtering (line 399: (w['end'] - w['start']) >=
min_window_duration) is a stable float threshold on DETERMINISTIC values (start/end are
frame/fps divisions, identical across runs).",
        "Float threshold safety: start/end times are computed as frame_index / fps + chu

```

ADVERSARIAL VERDICTS:

```

[
{
    "claim": "The COUNT of reels/highlight-segments produced by the configured+checkpointed
pipeline is DETERMINISTIC for a fixed (video, config, weights) ? safe to assert exactly (==),
not just approximately.",
    "verdict": "refuted",
    "evidence": "Adversarial code review of C:/Users/avidu/Projects/badminton-highlight-
indexer/.claude/worktrees/nightly-e2e found two categories of non-determinism:\n\n[1] DATABASE
INSERT FAILURE (SILENT):\nFile: backend/infrastructure/database.py:459-471,
add_play_segment()\n``python\nexcept Exception as e:\n    logger.error(f"Failed to add play
segment: {e}\")\n    return None\n``\nCan return None on ANY exception (DB locked, I/O,
constraint). Caller at motion.py:203 silently skips event creation if insert fails. Result:
segment count can vary if DB operations transiently fail.\n\n[2] RANDOM METADATA WITHOUT
SEEDING:\nFile: backend/pipeline/segmenters/motion.py:177-250, _populate_db_with_events()\nUses
unseeded stdlib random module (line 3: import random; no random.seed() call exists in entire
backend/)\n- Line 191: random.sample(player_pool, 2) \n- Line 196: random.randint(4, 25)\n-
Lines 206,212,216,218,225,230,238: Multiple random calls per segment\n\nComment at line 185
acknowledges: \"random call order here is load-bearing for reproducibility\" ? but
reproducibility depends on external seeding, which is absent.\n\n[3] SEGMENT DETECTION ITSELF IS
DETERMINISTIC:\nFile: backend/pipeline/segmenters/motion.py:39-175, _detect_motion_segments()\n-
Pure deterministic CV: cv2.absdiff(), cv2.threshold(), cv2.GaussianBlur(), signal smoothing,
thresholding\n- NO randomness in detection phase\n- Returns deterministic (start, end)
tuples\n\nCONTRADICTION: Segment count in play_segments table can vary due to transient DB
failures (Windows WAL contention per database.py:46-52 comment), despite deterministic
detection. Database operations are not wrapped in retry logic or transaction rollback guards
sufficient to ensure complete atomic insertion of all segments."
},

```

```
{
  "claim": "A full process->compile E2E can run in GitHub Actions (ubuntu, ci.yml env) with the
CPU motion segmenter on a tiny video WITHOUT any GPU/torch/model-weights/network, and the
resulting reel/segment COUNT is reproducible there.",
  "verdict": "refuted",
  "evidence": "The motion segmenter (backend/pipeline/segmenters/motion.py:1-256) has ZERO
dependencies on torch, CUDA, GPU, model weights, or network access?it uses only cv2, numpy,
scipy, stdlib random. Segment boundaries are determined by pure pixel-motion heuristics
(cv2.absdiff/threshold/dilate) which are fully deterministic, making segment counts
reproducible.\n\nHOWEVER, the claim conflates technical capability with tested reality. Evidence
adversarially found:\n\n1. **Code capability exists** ?\n  - Motion.py imports: cv2, time,
random, logging, typing (lines 1-8)\n  - Zero torch/cuda/gpu references anywhere in file\n  -
process_video() runs pure CV motion detection ? deterministic segment boundaries\n  - Tested
locally: segment count reproducible on identical inputs\n\n2. **CI environment supports running
it** ?\n  - .github/workflows/ci.yml:69-88 runs ubuntu-latest Python 3.13\n  - OpenCV + numpy
installed (runtime deps in pyproject.toml:30-31)\n  - torch installed CPU-only (line 84) but
motion segmenter never imports it\n\n3. **BUT NO E2E TEST EXISTS IN CI** ?\n  -
tests/test_motion_segmenter.py:1-223: ALL tests mock cv2 (line 113: patch.object)\n  -
tests/test_compiler.py:1-97: ALL tests mock subprocess.run (line 52, 75)\n  - No test runs
process_video on REAL video + compile_highlights with REAL ffmpeg\n  - run_all_tests.py (CI
entry point) runs the full suite but no true E2E exists\n\n4. **Untested integration risks
exist**:\n  - Motion segmenter untested on real videos in CI (mocked cv2 only)\n  -
VideoCompiler untested with real ffmpeg in CI (mocked subprocess only)\n  - Combined pipeline
never tested end-to-end in automated CI\n  - Any regress
```

2. assistant (2026-06-22T09:17:46.534Z)

I'll map the actual code at the worktree to ground the design in verified facts before synthesizing.

3. user (2026-06-22T09:17:48.698Z)

689c738 docs(pkg): mark P2c ? + the loop closed ? v1 LIGHT is feature-complete (#313)

```
---
README.md
__pycache__
bundle_ui.sh
doctor.py
install.py
nightly_regression.ps1
nightly_regression.py
register_nightly_task.ps1
run_phase3_eval.py
run_process_and_stitch.py
uninstall.py
```

4. user (2026-06-22T09:17:49.750Z)

```
total 12
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 .
drwxr-xr-x 1 avidu 197609    0 Jun 22 14:36 ..
-rw-r--r-- 1 avidu 197609 8357 Jun 22 14:36 ci.yml
---baselines---
experiment_leaderboard.json
fixtures
gemini_wrapper_2026-06-10.json
heuristic_lovo_n6.json
nightly_baseline.json
```

5. assistant (2026-06-22T09:17:53.086Z)

There's already a `nightly\_regression.py` and baselines. Let me read the existing pattern (PR #202) to reuse it.

6. user (2026-06-22T09:17:54.473Z)

```
1      """Nightly rally-quality regression tripwire.
2
3      Re-scores the **golden corpus** under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
4      seconds ? the WASB GPU trajectories are CACHED, only the windowing/scorer changes) and
diffs
5      the headline metrics against a FROZEN baseline
(``eval_baselines/nightly_baseline.json``).
6      Writes a dated report and exits non-zero on any regression beyond tolerance OR any per-
video
7      over-segmentation-guard regression.
8
9      This is the owner's dual-eval-set discipline made automatic (``memory/eval-dual-set-
regression``
10     + ``docs/R2_EVAL_METRIC_DESIGN.md`` §4.5): the growing GOLDEN-finalized set is re-run
every
11     night so a *late* change cannot silently lose an *earlier* win. It is **default-
ADDITIVE** ? it
12     does NOT change the promotion gate. It is a *tripwire*, not a re-training run: no GPU,
no Gemini,
13     no WASB re-extraction (those are cached and unchanged night to night).
14
15     Three tracked configs (all re-scored every night; a clean A/B/C on the same SERVED base
knobs):
16
17     * **DEFAULT** ? the SERVED production windowing, i.e. what users get today. Catches a
regression
18     to shipped behaviour. **NOTE (post-#204, 2026-06-18):** the R1 milestone is now
flipped ON, so
19     SERVED *is* cap=6 + R4 ? "default" now equals "milestone".
20     * **MILESTONE** ? the explicit R3+R4 config (cap=6 s + rally-END inactivity trim;
21     ``docs/QUALITY_ITERATIONS.md`` R3/R4). Now == DEFAULT (production shipped it); kept
explicit so
22     the nightly still names the milestone knobs and stays meaningful if production later
diverges.
23     * **BASELINE_OFF** ? the pre-R1-flip all-OFF windowing (every quality knob forced OFF).
The
24     permanent historical floor, so each report keeps showing the delta the milestone buys
(a launched
25     win must keep standing its ground, not be taken at launch-time value).
26
27     Ground truth + trajectories are LOCAL (the manifest points at owned, minors-free
trajectory CSVs
28     outside the repo + ``output/<name>_golden_features.json``); the corpus is NOT committed.
So the
29     baseline is tied to the owner's local golden corpus and is regenerated with ``--update-
baseline``
30     whenever the corpus grows or an intended improvement lands. (A cloud agent does not
suffice ? it
31     has the repo but not the cached trajectories; hence the local Windows Scheduled Task
wiring in
32     ``scripts/nightly_regression.ps1``.)
33
34     Usage
35     ----
36     Snapshot the baseline from current HEAD (after an intended improvement or a corpus
change):
37
38     python scripts/nightly_regression.py --update-baseline
39
40     Nightly check (exit 1 on regression; writes ``output/nightly_regression/<date>.md`` +
``.json``):
41
42     python scripts/nightly_regression.py
```

```

43
44     Exit codes
45     -----
46     * ``0`` ? all tracked configs clean (no regression), corpus matches the baseline.
47     * ``1`` ? at least one regression (an aggregate metric dropped beyond tolerance, or a
per-video
48         over-seg guard increased). Takes precedence over a stale baseline.
49     * ``2`` ? operational error (manifest/corpus missing, nothing scored).
50     * ``3`` ? no baseline to compare against (snapshot one with ``--update-baseline``).
51     * ``4`` ? baseline STALE (the corpus set or a config definition changed): a refresh is
needed,
52         but no regression was found on the comparable intersection.
53     """
54     from __future__ import annotations
55
56     import argparse
57     import datetime as _dt
58     import json
59     import os
60     import sys
61     from dataclasses import dataclass, field, replace
62     from typing import Any, Dict, List, Optional, Tuple
63
64     # ----- #
65     # Paths (resolved relative to the repo root, so the script is cwd-robust).
66     # ----- #
67     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
68     if REPO_ROOT not in sys.path:
69         sys.path.insert(0, REPO_ROOT) # importable as `python
scripts/nightly_regression.py` from anywhere
70     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "nightly_baseline.json")
71     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_regression")
72     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output", "human_lovo_manifest.json")
73     DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
74     DEFAULT_IOU = 0.5
75
76     # ----- #
77     # What we gate on (a tripwire ? focused, not every metric).
78     # * HIGHER-better aggregates regress when they DROP beyond ``metric_tol``.
79     # * merge_rate (lower-better) regresses when it RISES beyond ``rate_tol``.
80     # * per-video split_count / merge_count must NOT increase beyond ``over_seg_tol``
81     #   (the R2 §2.3.3 guard: "may not increase EITHER on ANY video").
82     # Boundary-error medians + seg_ratio are outlier-dominated diagnostics ? REPORTED, not
gated.
83     # ----- #
84     GATED_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
85     GATED_LOWER: Tuple[str, ...] = ("merge_rate",)
86     PER_VIDEO_GUARD: Tuple[str, ...] = ("split_count", "merge_count")
87
88     #: Aggregate metrics snapshotted into the baseline (the gated subset + report-only
context).
89     AGG_SNAPSHOT: Tuple[str, ...] = (
90         "tol_f1", "tol_precision", "tol_recall", "f1@0.5", "f1@0.25", "precision", "recall",
91         "merge_rate", "split_rate", "segment_count_ratio", "dstart_abs_median",
92         "dend_abs_median",
93     )
94     #: Per-video metrics snapshotted into the baseline.
95     PV_SNAPSHOT: Tuple[str, ...] = (
96         "tol_f1", "f1@0.5", "f1@0.25", "merge_rate", "split_count", "merge_count",
97         "num_pred", "num_gt",
98     )
99     DEFAULT_METRIC_TOL = 0.005 # F1-like drop that counts as a regression (absorbs float
jitter)
100    DEFAULT_RATE_TOL = 0.010 # merge_rate rise that counts as a regression

```

```

100     DEFAULT_OVER_SEG_TOL = 0      # per-video split/merge increase allowed (strict)
101
102
103     # ----- #
104     # The two tracked configs.
105     # ----- #
106     @dataclass(frozen=True)
107     class NightlyConfig:
108         """A named windowing config the nightly re-scores (preset + net-crossings gate)."""
109
110         name: str
111         preset: Any          # backend.eval.windowing.WindowingPreset
112         min_crossings: int = 0
113         note: str = ""
114
115
116     def tracked_configs() -> List[NightlyConfig]:
117         """DEFAULT (SERVED = shipped) + MILESTONE (explicit cap=6 + R4) + BASELINE_OFF (the
118 pre-flip floor).
119
120 All three are anchored on the SAME ``windowing.SERVED`` base knobs (velocity_thresh
121 / merge_gap /
122 min_window) so they differ ONLY in the quality knobs ? they are a clean A/B/C:
123
124 * **DEFAULT** ? ``SERVED`` verbatim, so it can never silently drift from the served
operating
125 point. **Post-#204 (2026-06-18) the R1 milestone is flipped ON ? SERVED IS cap=6 +
R4, so
126 DEFAULT now equals MILESTONE.** Its job: catch an *accidental* change to a
production windowing
127 knob.
128 * **MILESTONE** ? SERVED + the explicit R3/R4 knobs (``docs/QUALITY_ITERATIONS.md``
R3/R4). Pins
129 the milestone *definition* independent of what production ships, so "is cap=6 + R4
still scoring
130 what we expect?" stays answerable even if a future change moves DEFAULT.
131 * **BASELINE_OFF** ? SERVED with every quality knob forced OFF (the pre-R1-flip
windowing). Pins
132 the *historical floor* so each nightly report keeps showing the delta the
milestone buys (the
133 dual-eval-set discipline: a launched win must keep standing its ground, not be
taken at
134 launch-time value). It is the reference DEFAULT used to be before the flip."""
135 from backend.eval.windowing import SERVED
136
137     default = NightlyConfig(
138         name="default", preset=SERVED, min_crossings=0,
139         note="SERVED production windowing ? now the R1 milestone (cap=6 + R4) after the
#204 flip.",
140     )
141     milestone = NightlyConfig(
142         name="milestone",
143         preset=replace(SERVED, name="milestone", max_window_duration=6.0,
144             inactivity_end=1.5, inactivity_rally_thresh=0.20,
145             inactivity_min_density=0.30),
146         min_crossings=0,
147         note="R3 cap=6 s + R4 rally-END inactivity trim ? the shipped milestone (==
default post-#204).",
148     )
149     baseline_off = NightlyConfig(
150         name="baseline_off",
151         preset=replace(SERVED, name="baseline_off", max_window_duration=0.0,
152             hard_cap=False,
153             inactivity_end=0.0, inactivity_rally_thresh=0.08,
154             inactivity_min_density=0.34,

```

```

151             blob_fallback=False, blob_factor=4.0),
152         min_crossings=0,
153         note="Pre-R1-flip all-OFF windowing ? the historical floor; pins the value the
milestone buys.",
154     )
155     return [default, milestone, baseline_off]
156
157
158     # ----- #
159     # Tolerances bundle.
160     # ----- #
161     @dataclass(frozen=True)
162     class Tolerances:
163         metric_tol: float = DEFAULT_METRIC_TOL
164         rate_tol: float = DEFAULT_RATE_TOL
165         over_seg_tol: int = DEFAULT_OVER_SEG_TOL
166
167         def to_dict(self) -> Dict[str, Any]:
168             return {"metric_tol": self.metric_tol, "rate_tol": self.rate_tol,
169                     "over_seg_tol": self.over_seg_tol}
170
171
172     # ----- #
173     # Pure summary + comparison core (no I/O ? unit-testable).
174     # ----- #
175     def summarize_run(eval_result: Dict[str, Any]) -> Dict[str, Any]:
176         """Compact, comparable summary of one :func:`rally_seg_eval.evaluate` result.
177
178         Keeps only the deterministic quantities we compare ? aggregate MEANS (not the
bootstrap CI,
179         which is seeded but irrelevant to the gate) and per-video RAW metric values."""
180         agg = eval_result.get("aggregate", {}) or {}
181         aggregate = {k: (agg[k]["mean"] if isinstance(agg.get(k), dict) else None) for k in
AGG_SNAPSHOT}
182         per_video = {r["name"]: {k: r[k] for k in PV_SNAPSHOT if k in r} for r in
eval_result.get("rows", [])}
183         return {
184             "n": int(agg.get("n", len(eval_result.get("rows", [])))),
185             "preset": eval_result.get("preset"),
186             "min_crossings": eval_result.get("min_crossings", 0),
187             "iou": eval_result.get("iou", DEFAULT_IOU),
188             "aggregate": aggregate,
189             "per_video": per_video,
190         }
191
192
193     @dataclass
194     class Finding:
195         """One comparison outcome line (regression / improvement / stale note)."""
196
197         config: str
198         scope: str          # "aggregate" | "per_video" | "config"
199         metric: str
200         kind: str           # "regression" | "improvement" | "stale"
201         baseline: Optional[float] = None
202         current: Optional[float] = None
203         video: Optional[str] = None
204
205         @property
206         def delta(self) -> Optional[float]:
207             if self.baseline is None or self.current is None:
208                 return None
209             return self.current - self.baseline
210
211         def message(self) -> str:

```



```

212         loc = f"{self.config}/{self.video}/{self.metric}" if self.video else
f"{self.config}/{self.metric}"
213         if self.kind == "stale":
214             return f"[STALE] {self.config}: {self.metric}"
215         d = self.delta
216         ds = f"{d:+.4f}" if d is not None else "?"
217         b = "?" if self.baseline is None else f"{self.baseline:.4f}"
218         c = "?" if self.current is None else f"{self.current:.4f}"
219         tag = "REGRESSION" if self.kind == "regression" else "improvement"
220         return f"[{tag}] {loc}: {b} ? {c} ({ds})"
221
222
223     @dataclass
224     class ConfigComparison:
225         config: str
226         status: str = "ok" # "ok" | "regression" | "stale"
227         findings: List[Finding] = field(default_factory=list)
228         added_videos: List[str] = field(default_factory=list)
229         removed_videos: List[str] = field(default_factory=list)
230         aggregate_gated: bool = True # False when preset/corpus changed ? aggregate not
comparable
231
232         @property
233         def regressions(self) -> List[Finding]:
234             return [f for f in self.findings if f.kind == "regression"]
235
236         @property
237         def improvements(self) -> List[Finding]:
238             return [f for f in self.findings if f.kind == "improvement"]
239
240
241     def _preset_comparable(base_preset: Any, cur_preset: Any) -> bool:
242         """Are the two stored preset dicts the same windowing definition? (name is
cosmetic)."""
243         if not isinstance(base_preset, dict) or not isinstance(cur_preset, dict):
244             return base_preset == cur_preset
245         keys = set(base_preset) | set(cur_preset)
246         return all(base_preset.get(k) == cur_preset.get(k) for k in keys if k != "name")
247
248
249     def compare_config(name: str, base_sum: Dict[str, Any], cur_sum: Dict[str, Any],
250                       tols: Tolerances) -> ConfigComparison:
251         """Compare one config's current summary to its baseline summary.
252
253         * If the preset definition or the net-crossings gate changed, the whole config is
STALE
254         (numbers aren't comparable) ? flag it, skip gating.
255         * If the corpus (video set) changed, the aggregate is not comparable (different
videos) ?
256         skip the aggregate gate + flag added/removed, but STILL run the per-video gate on
the
257         INTERSECTION (a code regression on a shared video must still trip the wire).
258         """
259         cmp = ConfigComparison(config=name)
260
261         # 1) Config-definition drift ? stale, not gateable.
262         if not _preset_comparable(base_sum.get("preset"), cur_sum.get("preset")) \
263             or base_sum.get("min_crossings") != cur_sum.get("min_crossings"):
264             cmp.status = "stale"
265             cmp.aggregate_gated = False
266             cmp.findings.append(Finding(config=name, scope="config", metric="preset",
kind="stale"))
267             return cmp
268
269         base_pv = base_sum.get("per_video", {})

```

```

270     cur_pv = cur_sum.get("per_video", {})
271     base_names, cur_names = set(base_pv), set(cur_pv)
272     cmp.added_videos = sorted(cur_names - base_names)
273     cmp.removed_videos = sorted(base_names - cur_names)
274     corpus_changed = bool(cmp.added_videos or cmp.removed_videos)
275
276     # 2) Aggregate gate (only when the corpus matches ? means over different sets aren't
comparable).
277     if corpus_changed:
278         cmp.aggregate_gated = False
279         cmp.status = "stale"
280         cmp.findings.append(Finding(config=name, scope="config", metric="corpus",
kind="stale"))
281     else:
282         base_agg = base_sum.get("aggregate", {})
283         cur_agg = cur_sum.get("aggregate", {})
284         for m in GATED_HIGHER:
285             b, c = base_agg.get(m), cur_agg.get(m)
286             if b is None or c is None:
287                 continue
288             if c < b - tols.metric_tol:
289                 cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
290             elif c > b + tols.metric_tol:
291                 cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
292         for m in GATED_LOWER:
293             b, c = base_agg.get(m), cur_agg.get(m)
294             if b is None or c is None:
295                 continue
296             if c > b + tols.rate_tol:
297                 cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
298             elif c < b - tols.rate_tol:
299                 cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
300
301     # 3) Per-video over-seg guard on the intersection (always ? catches per-video code
regressions).
302     for vid in sorted(base_names & cur_names):
303         for m in PER_VIDEO_GUARD:
304             b, c = base_pv[vid].get(m), cur_pv[vid].get(m)
305             if b is None or c is None:
306                 continue
307             if c > b + tols.over_seg_tol:
308                 cmp.findings.append(Finding(name, "per_video", m, "regression",
float(b), float(c), video=vid))
309
310     if cmp.regressions:
311         cmp.status = "regression"
312     elif cmp.status != "stale":
313         cmp.status = "ok"
314     return cmp
315
316
317 @dataclass
318 class NightlyResult:
319     comparisons: List[ConfigComparison] = field(default_factory=list)
320     skipped: Dict[str, List[str]] = field(default_factory=dict) # config -> manifest
videos not scored
321
322     @property
323     def has_regression(self) -> bool:
324         return any(c.status == "regression" for c in self.comparisons)
325
326     @property
327     def has_stale(self) -> bool:
328         return any(c.status == "stale" for c in self.comparisons)
329

```

```

330     def overall_status(self) -> str:
331         if self.has_regression:
332             return "REGRESSION"
333         if self.has_stale:
334             return "STALE"
335         return "PASS"
336
337     def exit_code(self) -> int:
338         if self.has_regression:
339             return 1
340         if self.has_stale:
341             return 4
342         return 0
343
344
345     def compare_all(baseline: Dict[str, Any], current: Dict[str, Any],
346                    tols: Tolerances, skipped: Optional[Dict[str, List[str]]] = None) ->
NightlyResult:
347         """Compare every tracked config's current summary to the baseline summary."""
348         res = NightlyResult(skipped=skipped or {})
349         base_cfgs = baseline.get("configs", {})
350         cur_cfgs = current.get("configs", {})
351         for name in cur_cfgs:
352             if name not in base_cfgs:
353                 cmp = ConfigComparison(config=name, status="stale", aggregate_gated=False)
354                 cmp.findings.append(Finding(config=name, scope="config",
metric="missing_in_baseline", kind="stale"))
355                 res.comparisons.append(cmp)
356                 continue
357             res.comparisons.append(compare_config(name, base_cfgs[name], cur_cfgs[name],
tols))
358         return res
359
360
361     # ----- #
362     # Report formatting (pure).
363     # ----- #
364     def _fmt(v: Optional[float]) -> str:
365         return "?" if v is None else f"{v:.4f}"
366
367
368     def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
369                      result: NightlyResult, tols: Tolerances, date: str) -> str:
370         L: List[str] = []
371         status = result.overall_status()
372         badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION", "STALE": "? STALE (baseline
refresh needed)"}[status]
373         L.append(f"# Nightly rally-quality regression ? {date}")
374         L.append("")
375         L.append(f"***Status: {badge}** · exit code {result.exit_code()}")
376         L.append("")
377         L.append(f"- HEAD: `{current.get('git_commit', '?')}` · baseline: "
378                f"`{baseline.get('git_commit', '?')}` (snapshot
{baseline.get('generated_at', '?')})")
379         L.append(f"- Corpus manifest: `{current.get('manifest', '?')}` ·
IoU={current.get('iou', DEFAULT_IOU)}")
380         L.append(f"- Tolerances: metric ±{tols.metric_tol}, rate ±{tols.rate_tol}, "
381                f"f"per-video over-seg ±{tols.over_seg_tol}")
382         L.append("")
383
384         # Up-front regression / stale summary.
385         regs = [f for c in result.comparisons for f in c.regressions]
386         if regs:
387             L.append("## ? Regressions")
388             L.append("")

```

```

389         for f in regs:
390             L.append(f"- {f.message()}")
391         L.append("")
392     if result.has_stale:
393         L.append("## ? Stale / not comparable")
394         L.append("")
395         for c in result.comparisons:
396             if c.status == "stale":
397                 if c.added_videos:
398                     L.append(f"- `{c.config}`: corpus added {c.added_videos}")
399                 if c.removed_videos:
400                     L.append(f"- `{c.config}`: corpus removed {c.removed_videos}")
401                 if not c.added_videos and not c.removed_videos:
402                     L.append(f"- `{c.config}`: config definition changed vs baseline ?
re-snapshot with --update-baseline")
403                 L.append(f"- Action: `python scripts/nightly_regression.py --update-baseline`
(after confirming the change is intended).")
404                 L.append("")
405
406         # Skipped (no silent caps).
407         skipped_any = {k: v for k, v in result.skipped.items() if v}
408         if skipped_any:
409             L.append("## ? Skipped manifest videos (missing trajectory or GT ? NOT silently
dropped)")
410             L.append("")
411             for cfg, vids in skipped_any.items():
412                 L.append(f"- `{cfg}`: {vids}")
413             L.append("")
414
415         # Per-config detail.
416         cur_cfgs = current.get("configs", {})
417         base_cfgs = baseline.get("configs", {})
418         for cmp in result.comparisons:
419             cur = cur_cfgs.get(cmp.config, {})
420             base = base_cfgs.get(cmp.config, {})
421             L.append(f"## Config `{cmp.config}` ? {cur.get('note', '')}")
422             L.append("")
423             L.append(f"_status: {cmp.status}; n={cur.get('n', '?')} (baseline
n={base.get('n', '?')})")
424             L.append("")
425             cagg = cur.get("aggregate", {})
426             bagg = base.get("aggregate", {})
427             L.append("| metric | baseline | current | ? | gated |")
428             L.append("|---|---|---|---|---|")
429             for m in AGG_SNAPSHOT:
430                 b, c = bagg.get(m), cagg.get(m)
431                 d = (c - b) if (b is not None and c is not None) else None
432                 gated = "?" if (m in GATED_HIGHER or m in GATED_LOWER) and
cmp.aggregate_gated else ""
433                 L.append(f"| {m} | {_fmt(b)} | {_fmt(c)} | {_fmt(d)} | {gated} |")
434             L.append("")
435             # Per-video over-seg guard table.
436             cpv = cur.get("per_video", {})
437             bpv = base.get("per_video", {})
438             names = sorted(set(cpv) | set(bpv))
439             if names:
440                 L.append("| video | tolF1 (b?c) | split (b?c) | merge (b?c) |")
441                 L.append("|---|---|---|---|")
442                 for v in names:
443                     cb, cc = bpv.get(v, {}), cpv.get(v, {})
444                     tf = f"{_fmt(cb.get('tol_f1'))}?{_fmt(cc.get('tol_f1'))}"
445                     sp = f"{cb.get('split_count', '?')}?{cc.get('split_count', '?')}"
446                     mg = f"{cb.get('merge_count', '?')}?{cc.get('merge_count', '?')}"
447                     flag = " ?" if any(f.video == v and f.kind == "regression" for f in
cmp.findings) else ""

```

```

448         L.append(f"| {v}{flag} | {tf} | {sp} | {mg} |")
449         L.append("")
450         impls = cmp.improvements
451         if impls:
452             L.append("_Improvements over baseline (consider `--update-baseline` to
ratchet the floor up):_")
453             for f in impls:
454                 L.append(f"- {f.message()}")
455             L.append("")
456
457         L.append("----")
458         L.append("_Tripwire only ? re-scores CACHED trajectories (no GPU/Gemini/WASB). Does
not change "
459             "the promotion gate. See docs/R2_EVAL_METRIC_DESIGN.md + memory/eval-dual-
set-regression._")
460         return "\n".join(L)
461
462
463     # ----- #
464     # I/O shell.
465     # ----- #
466     def _git_commit() -> str:
467         import subprocess
468         try:
469             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
capture_output=True, text=True, timeout=10)
470             return out.stdout.strip() or "?"
471         except Exception:
472             return "?"
473
474
475     def _display_path(path: str) -> str:
476         """A clean, portable form for the committed baseline: ``output/<file>`` when the
path lives
477         in an ``output`` dir (the production layout), else repo-relative, else the path as
given.
478         Metadata only ? the comparison never reads it."""
479         norm = path.replace("\\", "/")
480         parent = os.path.basename(os.path.dirname(norm))
481         if parent == "output":
482             return "output/" + os.path.basename(norm)
483         try:
484             rel = os.path.relpath(path, REPO_ROOT)
485             if not rel.startswith(".."):
486                 return rel.replace("\\", "/")
487         except ValueError:
488             pass
489         return norm
490
491
492     def run_corpus(configs: List[NightlyConfig], manifest: str, features_dir: str,
493                   iou: float = DEFAULT_IOU) -> Tuple[Dict[str, Any], Dict[str, List[str]]]:
494         """Score every tracked config over the golden corpus. Returns (snapshot, skipped-
per-config).
495         ``snapshot`` is the serialisable structure stored as / compared to the baseline.
496         ``skipped``
497         lists manifest videos NOT scored (missing trajectory or empty GT) per config ?
498         surfaced in
499         the report so the corpus can't silently shrink."""
500         from backend.eval import rally_seg_eval
501
502         golden = rally_seg_eval.load_golden(manifest, features_dir)
503         manifest_names = [v["name"] for v in golden]
504         if not golden:

```

```

505         raise RuntimeError(f"no golden videos loaded from {manifest}")
506
507     snapshot: Dict[str, Any] = {"manifest": _display_path(manifest), "iou": iou,
"configs": {}}
508     skipped: Dict[str, List[str]] = {}
509     scored_any = False
510     for cfg in configs:
511         result = rally_seg_eval.evaluate(golden, cfg.preset, iou=iou,
min_crossings=cfg.min_crossings)
512         summary = summarize_run(result)
513         summary["note"] = cfg.note
514         snapshot["configs"][cfg.name] = summary
515         scored = set(summary["per_video"])
516         skipped[cfg.name] = [n for n in manifest_names if n not in scored]
517         scored_any = scored_any or bool(scored)
518     if not scored_any:
519         raise RuntimeError(
520             f"scored 0 videos for every config ? are the trajectory CSVs in {manifest}
present "
521             f"and do the {features_dir}/<name>_golden_features.json files have 'gts'?"")
522     return snapshot, skipped
523
524
525     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
526         if not os.path.exists(path):
527             return None
528         with open(path, encoding="utf-8") as fh:
529             return json.load(fh)
530
531
532     def save_baseline(path: str, snapshot: Dict[str, Any], tols: Tolerances) -> None:
533         out = dict(snapshot)
534         out["generated_at"] = _dt.date.today().isoformat()
535         out["git_commit"] = _git_commit()
536         out["tolerances"] = tols.to_dict()
537         out["_doc"] = ("Frozen nightly rally-quality regression baseline. Regenerate with "
538             "`python scripts/nightly_regression.py --update-baseline` after an
intended "
539             "improvement or a corpus change. Tied to the LOCAL golden corpus (not
committed).")
540         os.makedirs(os.path.dirname(path), exist_ok=True)
541         tmp = path + ".tmp"
542         with open(tmp, "w", encoding="utf-8") as fh:
543             json.dump(out, fh, indent=2, sort_keys=True)
544             fh.write("\n")
545         os.replace(tmp, path)
546
547
548     def build_parser() -> argparse.ArgumentParser:
549         p = argparse.ArgumentParser(description="Nightly rally-quality regression tripwire
(offline, CPU).")
550         p.add_argument("--manifest", default=DEFAULT_MANIFEST, help="golden manifest JSON")
551         p.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR,
552             help="dir holding <name>_golden_features.json")
553         p.add_argument("--baseline", default=DEFAULT_BASELINE, help="frozen baseline JSON
path")
554         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR, help="where dated reports
are written")
555         p.add_argument("--iou", type=float, default=DEFAULT_IOU)
556         p.add_argument("--metric-tol", type=float, default=DEFAULT_METRIC_TOL,
557             help="F1-like aggregate drop that counts as a regression")
558         p.add_argument("--rate-tol", type=float, default=DEFAULT_RATE_TOL,
559             help="merge_rate rise that counts as a regression")
560         p.add_argument("--over-seg-tol", type=int, default=DEFAULT_OVER_SEG_TOL,
561             help="per-video split/merge-count increase allowed before it's a

```



```

618         log.info("report written to %s", md_path)
619
620     print(report)
621     code = result.exit_code()
622     if code == 1:
623         log.error("NIGHTLY REGRESSION DETECTED (%d) ? see %s",
624                 sum(len(c.regressions) for c in result.comparisons), args.report_dir)
625     elif code == 4:
626         log.warning("baseline STALE ? corpus/config changed; refresh with --update-
baseline")
627     else:
628         log.info("nightly regression check PASSED")
629     return code
630
631
632 if __name__ == "__main__":
633     raise SystemExit(main())
634

```

7. user (2026-06-22T09:17:54.523Z)

```

1      <#
2      .SYNOPSIS
3          Nightly rally-quality regression tripwire ? wrapper for the Windows Scheduled Task.
4
5      .DESCRIPTION
6          Re-scores the golden corpus under the current checkout via
scripts/nightly_regression.py
7          (CPU, seconds ? the WASB trajectories are CACHED) and diffs vs the frozen baseline.
Writes a
8          dated report + log under output/nightly_regression/ and a LATEST_STATUS.txt marker,
then exits
9          with the checker's exit code so Task Scheduler records PASS / REGRESSION / STALE.
10
11         GRACEFUL no-op (exit 0) when the checker or the local golden corpus isn't present
yet (e.g.
12         before this PR is merged into the checkout the task points at) ? so a registered
task never
13         errors in the interim and self-activates once the files land.
14
15         Cloud agents can't do this: the cached trajectories + golden features live OUTSIDE
the repo on
16         this machine. Hence a LOCAL scheduled task, not a CronCreate/cloud routine.
17
18     .PARAMETER RepoRoot
19         Repo root to test. Default: the parent of this script's directory.
20
21     .PARAMETER PythonExe
22         Python interpreter. Default: 'python'.
23
24     .PARAMETER Pull
25         If set, 'git pull --ff-only' before scoring (off by default ? never auto-touch a
shared checkout
26         that may hold a sibling session's WIP).
27     #>
28     [CmdletBinding()]
29     param(
30         [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),
31         [string]$PythonExe = 'python',
32         [switch]$Pull
33     )
34
35     $ErrorActionPreference = 'Continue'
36     $date = Get-Date -Format 'yyyy-MM-dd'

```



```

37     $reportDir = Join-Path $RepoRoot 'output\nightly_regression'
38     $logDir = Join-Path $reportDir 'logs'
39     New-Item -ItemType Directory -Force -Path $logDir | Out-Null
40     $log = Join-Path $logDir "$date.log"
41     $statusFile = Join-Path $reportDir 'LATEST_STATUS.txt'
42
43     function Write-Status([string]$status, [int]$code, [string]$detail) {
44         $stamp = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
45         "$stamp status=$status exit=$code $detail" | Out-File -FilePath $statusFile
46     }
47
48     $checker = Join-Path $RepoRoot 'scripts\nightly_regression.py'
49     $manifest = Join-Path $RepoRoot 'output\human_lovo_manifest.json'
50
51     # Graceful no-op until the checker + corpus are present (pre-merge / corpus absent).
52     if (-not (Test-Path $checker)) {
53         "$[date] nightly_regression.py not present at $checker ? skipping (awaiting merge)."
54     } | Tee-Object -FilePath $log
55     Write-Status 'SKIPPED' 0 'checker absent (awaiting merge)'
56     exit 0
57
58     if (-not (Test-Path $manifest)) {
59         "$[date] golden manifest not present at $manifest ? skipping (corpus absent on this
60 machine)." | Tee-Object -FilePath $log
61     }
62     Write-Status 'SKIPPED' 0 'golden corpus absent'
63     exit 0
64
65     Set-Location $RepoRoot
66     if ($Pull) {
67         "$[date] git pull --ff-only" | Tee-Object -FilePath $log -Append
68         & git -C $RepoRoot pull --ff-only 2>&1 | Tee-Object -FilePath $log -Append
69     }
70
71     $head = (& git -C $RepoRoot rev-parse --short HEAD 2>$null)
72     "$[date] nightly rally-quality regression ? HEAD $head" | Tee-Object -FilePath $log
73 -Append
74
75     & $PythonExe $checker 2>&1 | Tee-Object -FilePath $log -Append
76     $code = $LASTEXITCODE
77
78     switch ($code) {
79         0 { Write-Status 'PASS' 0 "report: $reportDir\$date.md" }
80         1 { Write-Status 'REGRESSION' 1 "report: $reportDir\$date.md"
81         "$[date] *** NIGHTLY REGRESSION DETECTED ? see $reportDir\$date.md ***" | Tee-
82 Object -FilePath $log -Append }
83         4 { Write-Status 'STALE' 4 "baseline refresh needed ? see $reportDir\$date.md" }
84         default { Write-Status 'ERROR' $code "checker error ? see $log" }
85     }
86
87     exit $code
88

```

8. assistant (2026-06-22T09:17:58.755Z)

This is the PR #202 pattern. Now let me verify the key code paths: CLI single-shot mode, motion segmenter's random usage, the compiler, and the CI workflow.

9. user (2026-06-22T09:18:00.645Z)

```

990     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
991     # the executor is in-process. Mark them failed so pollers see a terminal state.
992     swept = db_instance.fail_stale_jobs()
993     if swept:

```

```

994         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
995
996         # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
997         if args.video_path and not args.server and not args.app:
998             print(f"[CLI] Processing video file: {args.video_path}")
999             # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
1000             # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
1001             # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1002             storage_cfg = getattr(global_config, "storage", None)
1003             try:
1004                 input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1005             except ValueError as e:
1006                 print(f"[FAIL] {e}")
1007                 return
1008             if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1009                 print(f"[FAIL] Video file not found: {args.video_path}")
1010                 return
1011             local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1012
1013             video_id = compute_video_id(local_video)
1014             was_reingest = db_instance.get_video(video_id) is not None
1015
1016             # Validate (with-context variant surfaces ffprobe_meta for the report)
1017             print("[CLI] Validating...")
1018             val_results, val_context = registry.run_all_with_context(local_video,
global_config)
1019             failures = [r for r in val_results if not r.passed]
1020
1021             # Save video status
1022             status = "failed" if failures else "processed"
1023             db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
1024
1025             seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
1026             segmenter_actual = None
1027             segmentation_ran = False
1028             try:
1029                 print("\n" + "="*40)
1030                 print("VALIDATION SUMMARY")
1031                 print("="*40)
1032                 for r in val_results:
1033                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
1034                     print(f"{status_symbol} {r.validator_name}: {r.message}")
1035                 print("="*40 + "\n")
1036
1037                 if failures:
1038                     print("[FAIL] Video failed checks. Indexing halted.")
1039                     return
1040
1041             # Index
1042             segmenter_cls = segmenter_registry.get(seg_name)
1043             if not segmenter_cls:
1044                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
1045                 return
1046
1047             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1048             segmenter_actual = seg_name
1049             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1050             segmenter = segmenter_cls(db_instance, global_config)

```

```

1051         result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
1052         segmentation_ran = True
1053
1054         if isinstance(result, Failure):
1055             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1056             print("[FAIL] Indexing halted for video.")
1057             segments = []
1058         else:
1059             segments = result.value if hasattr(result, "value") else result
1060             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
1061             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1062             _finalize_reingest(video_id, segments, play_wm, cand_wm)
1063         finally:
1064             # Always emit the per-video observability report. Never raises.
1065             paths = emit_indexer_report(
1066                 db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
1067                 val_results=val_results, val_context=val_context,
1068                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
1069                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
1070             )
1071             if paths:
1072                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1073                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1074                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
1075             return
1076
1077     # Start Local Server Mode
1078     if args.app or args.server or not args.video_path:
1079         if args.app:

```

10. user (2026-06-22T09:18:01.308Z)

```

1     import cv2
2     import time
3     import random
4     from typing import List, Dict, Any, Tuple
5     import logging
6     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
7     from backend.results import Success, Failure # standardized contract
8
9     logger = logging.getLogger(__name__)
10
11     class MotionPlaySegmenter(BasePlaySegmenter):
12         @property
13         def name(self) -> str:
14             return "motion"
15
16         @property
17         def description(self) -> str:
18             return "Traditional CV pixel-motion heuristics tracking frame differences."
19
20         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21             """
22             Processes a video to detect play segments using motion heuristics,
23             populates SQLite with detected segments, and indexes detailed events.
24             """
25             if not player_pool:
26                 player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28             cap = cv2.VideoCapture(video_path)

```

```

29         if not cap.isOpened():
30             logger.error(f"Segmenter could not open video: {video_path}")
31             return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33         final_segments = self._detect_motion_segments(cap, max_frames)
34
35         segments_data = self._populate_db_with_events(video_id, final_segments,
player_pool)
36
37         return Success(value=segments_data)
38
39     def _detect_motion_segments(self, cap: Any, max_frames: int = None) ->
List[Tuple[float, float]]:
40         """Run the pixel-motion heuristic over ``cap`` and return the detected
41         (start, end) play segments.
42
43         Reads/sub-samples frames, builds a per-frame motion signal, smooths it,
44         thresholds it into raw segments, then merges dead-time gaps and filters
45         out segments shorter than ``min_segment_duration``. ``cap`` is released
46         before returning. This is the detection half of ``process_video`` ? it
47         performs no DB writes and no fabrication.
48         """
49         fps = cap.get(cv2.CAP_PROP_FPS)
50         total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
51         duration = total_frames / fps if fps > 0 else 0
52
53         # Sub-sample settings (5 FPS for analysis)
54         analysis_fps = 5.0
55         frame_interval = max(1, int(fps / analysis_fps))
56
57         idx_cfg = self.config.get("indexing", {})
58         motion_threshold = idx_cfg.get("motion_threshold", 0.08)
59         min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
60         dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)

```

11. assistant (2026-06-22T09:18:05.132Z)

Now let me confirm the random usage in `\_populate\_db\_with\_events` (the count-affecting question) and check the CI workflow + DB count semantics.

12. user (2026-06-22T09:18:06.562Z)

```

175         return final_segments
176
177     def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]],
178                                player_pool: List[str]) -> List[Dict[str, Any]]:
179         """Persist ``final_segments`` and their fabricated metadata/events to the DB.
180
181         For each detected segment this assigns a random server/receiver, fabricates
182         rich tags, then indexes a serve / 0?3 smash / termination event set ? all
183         with the stdlib ``random`` module (the legacy demo's fabricated baseline).
184         Returns the per-segment dicts handed back to the caller. The ``random``
185         call order here is load-bearing for reproducibility and must not change.
186         """
187         # Populate SQLite DB with segments and metadata events
188         segments_data = []
189         for start, end in final_segments:
190             # Assign Server and Receiver randomly from pool
191             server, receiver = random.sample(player_pool, 2)
192
193             # Formulate dynamic rich tags
194             dur = end - start
195             metadata = {

```

```

196         "shot_count": random.randint(4, 25),
197         "intensity": "high" if dur > 12 else "medium",
198         "avg_speed_kmh": round(random.uniform(40, 85), 1)
199     }
200
201     segment_id = self.db.add_play_segment(video_id, start, end, server,
receiver, metadata)
202
203     if segment_id:
204         # Add sub-segment events
205         # A: Serve Event (always at start)
206         serve_foul = random.random() < 0.15 # 15% chance of serve foul
207         self.db.add_event(
208             segment_id=segment_id,
209             timestamp=start + 0.5,
210             event_type="serve",
211             player=server,
212             details={"status": "foul" if serve_foul else "valid", "type":
"low_serve" if random.random() > 0.5 else "high_serve"}
213         )
214
215         # B: Smash Events (1 to 3 random smashes per segment)
216         smashes_count = random.randint(0, 3)
217         for _ in range(smashes_count):
218             smash_time = random.uniform(start + 1.5, end - 1.0)
219             smasher = random.choice([server, receiver])
220             self.db.add_event(
221                 segment_id=segment_id,
222                 timestamp=round(smash_time, 2),
223                 event_type="smash",
224                 player=smasher,
225                 details={"speed_kmh": round(random.uniform(120, 250), 1)}
226             )
227
228         # C: Segment Termination Event (foul/winner)
229         foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
230         is_foul = random.random() > 0.4
231         error_player = random.choice([server, receiver])
232
233         self.db.add_event(
234             segment_id=segment_id,
235             timestamp=end - 0.2,
236             event_type="foul" if is_foul else "winner",
237             player=error_player,
238             details={"type": random.choice(foul_types)} if is_foul else
{"shot_type": "drop" if random.random() > 0.5 else "smash"}
239         )
240
241         # Store segment representation to return to caller
242         segments_data.append({
243             "id": segment_id,
244             "start_time": start,
245             "end_time": end,
246             "duration": dur,
247             "server": server,
248             "receiver": receiver,
249             "metadata": metadata
250         })
251
252     return segments_data
253
254     # Register the segmenter

```

```

1     name: CI
2
3     on:
4         push:
5             branches: [master]
6         pull_request:
7
8     permissions:
9         contents: read
10
11    concurrency:
12        group: ci-${{ github.ref }}
13        cancel-in-progress: true
14
15    jobs:
16        # Lint gate (ruff.toml at repo root): pyflakes/pycodestyle + bugbear.
17        # Also the repo-wide syntax guard ? ruff reports invalid-syntax on every
18        # tracked .py, including scripts/ and training/, which the backend-only
19        # py_compile smoke sweep does not reach.
20        lint:
21            name: Ruff lint
22            runs-on: ubuntu-latest
23            steps:
24                - uses: actions/checkout@v6
25                - uses: actions/setup-python@v6
26                  with:
27                      python-version: "3.13"
28                - name: Install ruff
29                  run: python -m pip install ruff
30                - name: Ruff check
31                  run: ruff check . --output-format github
32
33        # Type gate (mypy.ini at repo root): lenient green baseline with an explicit
34        # per-module quarantine list ? ratchet by removing entries as modules clean
35        # up. Typed core deps are installed so mypy sees their real signatures
36        # (without them ignore_missing_imports would Any-stub half the checks).
37        typecheck:
38            name: Mypy (lenient baseline)
39            runs-on: ubuntu-latest
40            steps:
41                - uses: actions/checkout@v6
42                - uses: actions/setup-python@v6
43                  with:
44                      python-version: "3.13"
45                - name: Install mypy + typed core deps
46                  run: python -m pip install mypy pydantic fastapi uvicorn python-dotenv google-
47genai numpy
48                - name: Mypy
49                  run: mypy backend training
50
51        # Fast guard against the 7fbb0 class of breakage: a SyntaxError or
52        # module-level import error shipping while the suite stays green.
53        # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold
54        # on machines without the GPU stack (test_import_smoke stubs what's absent).
55        import-smoke:
56            name: Syntax sweep + import smoke (no GPU stack)
57            runs-on: ubuntu-latest
58            steps:
59                - uses: actions/checkout@v6
60                - uses: actions/setup-python@v6
61                  with:
62                      python-version: "3.13"
63                - name: Install minimal deps
64                  run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
65genai

```

```

64         - name: Run import smoke tests
65         run: python -m pytest tests/test_import_smoke.py -v
66
67     full-suite:
68         name: Full test suite (offline, mocked)
69         runs-on: ubuntu-latest
70         steps:
71             - uses: actions/checkout@v6
72             - uses: actions/setup-python@v6
73               with:
74                 python-version: "3.13"
75                 cache: pip
76             - name: System libs for opencv-python
77               run: |
78                 sudo apt-get update
79                 sudo apt-get install -y libgl1 libglu1-mesa-dev || sudo apt-get install -y
libgl1 libglu1-mesa-dev
80             - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
81               # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
82               # need an out-of-band index that PEP 621 can't express. Keep installing
83               # them separately here, before the editable install pulls the rest.
84               run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
85             - name: Install package (editable) + dev tooling
86               # Editable PEP 621 install: runtime deps + the `dev` group
87               # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
88               run: python -m pip install -e .[dev]
89               # obsreport (private, soft dep) is absent here: reporting tests skip via
90               # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
91               # is therefore calibrated against THIS lane's number (local runs with
92               # obsreport installed measure a little higher).
93               # Floor calibrated 2026-06-11 against this lane's measured 72% (local
94               # with obsreport: 73%). Ratchet it UP as coverage grows; never down
95               # without an owner decision.
96             - name: Run full suite (with coverage floor)
97               run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
98
99         # Cross-OS determinism guard for the committed golden regression fixtures
100         # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
101         # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
102         # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
103         # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
104         # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
105         golden-crossos:
106             name: Golden fixtures cross-OS (${{ matrix.os }})
107             strategy:
108                 fail-fast: false
109             matrix:
110                 os: [ubuntu-latest, windows-latest, macos-latest]
111             runs-on: ${{ matrix.os }}
112             steps:
113                 - uses: actions/checkout@v6
114                 - uses: actions/setup-python@v6
115                   with:
116                     python-version: "3.13"
117                 - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
118                   run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
119                 - name: Golden-fixtures characterization (cross-OS determinism guard)
120                   # Also runs the freeze-real / time-origin-reset / M2 refusal-gate tests
121                   # tmp_path only ? no GPU/network) so the reset metric-invariance (fix #2) and
122                   # de-attribution paths get cross-OS coverage and surface any win/mac float

```

```

drift.
123         run: |
124             python -m backend.eval.golden_fixtures --check
125             python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
126
127         # Commercial-clean licensing guardrail (MIT/Apache/BSD only ? PACKAGING_PLAN.md P0d).
Scans the
128         # RUNTIME dependency closure (`pip install .` ? no torch/dev) and fails on any
copyleft
129         # (GPL/AGPL/LGPL/SSPL). torch (BSD-3) is installed out-of-band and is not in this
closure. This
130         # automates a guardrail that was human-only; the report is printed for review either
way.
131         license-scan:
132             name: Dependency license scan (no copyleft in runtime deps)
133             runs-on: ubuntu-latest
134             steps:
135                 - uses: actions/checkout@v6
136                 - uses: actions/setup-python@v6
137                 with:
138                     python-version: "3.13"
139                 - name: Install runtime deps + pip-licenses
140                     run: |
141                         python -m pip install .
142                         python -m pip install pip-licenses
143                 - name: Report dependency licenses
144                     run: python -m piplicenses --format=markdown --with-urls --order=license
145                 - name: Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the runtime closure
146                     run: |
147                         if python -m piplicenses --format=plain | grep -iE "GPL|SSPL"; then
148                             echo "::error::A runtime dependency carries a copyleft (GPL/AGPL/LGPL/SSPL)
license ? the guardrail is MIT/Apache/BSD only. Review the report above."
149                             exit 1
150                         fi
151                     echo "OK: no GPL/AGPL/LGPL/SSPL in the runtime dependency closure."
152
153         # Built-wheel smoke (PACKAGING_PLAN.md P0d): build the wheel, install it NON-editable
from a
154         # non-repo cwd, and launch the console script. Catches packaging regressions the
source suite
155         # can't ? namespace-package resolution (backend has no __init__.py), missing data
files, hidden
156         # imports. (Validated locally: a clean-venv wheel install + CPU torch runs `indexer
--help`.)
157         build-artifact:
158             name: Built wheel smoke (install + launch)
159             runs-on: ubuntu-latest
160             steps:
161                 - uses: actions/checkout@v6
162                 - uses: actions/setup-python@v6
163                 with:
164                     python-version: "3.13"
165                 - name: System libs for opencv-python
166                     run: |
167                         sudo apt-get update
168                         sudo apt-get install -y libgl1 libglu1 libglu2.0-0t64 || sudo apt-get install -y
libgl1 libglu1 libglu2.0-0
169                 - name: Build the wheel
170                     run: |
171                         python -m pip install build
172                         python -m build --wheel
173                 - name: Install the built wheel + CPU torch (out-of-band index)
174                     run: |
175                         python -m pip install torch torchvision --index-url

```



```

https://download.pytorch.org/whl/cpu
176         python -m pip install dist/*.whl
177     - name: Launch the installed console script from a non-repo dir
178         run: |
179             cd /tmp
180             indexer --help
181             python -c "import backend, training, backend.utils.app_home; print('wheel
import OK')"
182

```

14. assistant (2026-06-22T09:18:18.698Z)

I now have what I need: the existing PR #202 pattern (quality tripwire on cached trajectories), the CLI single-shot path, the motion segmenter's unseeded `random` (count-unaffected ? verified below), the compiler path, and the CI workflow with its golden-crossos pattern. Let me confirm one critical determinism point: whether `random` affects the COUNT.

The `random` calls in `\_populate\_db\_with\_events` (motion.py:191-238) generate metadata/events only ? they do NOT change the number of `final\_segments` iterated (that comes from `\_detect\_motion\_segments`, pure CV). The `if segment\_id:` guard at line 203 only skips on a `None` insert (DB failure). So COUNT is deterministic modulo transient DB-insert failure. Let me verify the over-seg/min_shot_count compile filter is the only count-affecting filter downstream.

15. user (2026-06-22T09:18:20.577Z)

```

705     @app.post("/api/compile")
706     def compile_highlights(req: CompileRequest):
707         video = db_instance.get_video(req.video_id)
708         if not video:
709             raise HTTPException(status_code=404, detail="Indexed video record not found.")
710
711         # Retrieve matching play segments from db
712         all_segments = db_instance.query_play_segments(req.video_id, {})
713         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
714
715         if not matched_segments:
716             raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
717
718         # Reject path traversal / absolute output names (os.path.join would otherwise let an
719         # absolute or '..'-laden output_filename write anywhere on disk).
720         try:
721             out_name = safe_output_filename(req.output_filename)
722         except ValueError as e:
723             raise HTTPException(status_code=400, detail=str(e)) from e
724
725         # The configured shot-count floor (stitching.min_shot_count) applies on the API
726         # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
727         # metadata are exempt: the floor filters known counts only, so explicitly
728         # requested manual/legacy segments can't silently vanish under the default floor.
729         stitching = getattr(global_config, "stitching", None) or StitchingConfig()
730         kept = []
731         for s in matched_segments:
732             meta = s.get("metadata") or {}
733             sc = meta.get("shot_count") if isinstance(meta, dict) else None
734             try:
735                 if sc is not None and int(sc) < stitching.min_shot_count:
736                     continue
737             except (TypeError, ValueError):
738                 pass # unparseable count -> treat as unknown, keep
739             kept.append(s)
740         if len(kept) < len(matched_segments):
741             logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "

```

```

742             f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
743         if not kept:
744             raise HTTPException(status_code=400,
745                                 detail=f"All {len(matched_segments)} matching segments fall
below "
746                                     f"stitching.min_shot_count={stitching.min_shot_count}.")
747
748         # Extract start/end time pairs
749         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
750
751         # Run compiler with the configured stitching paddings + optional branding watermark,
localized to
752         # the UI locale the SPA rendered in (req.locale): the reel's watermark reads in the
user's language
753         # with a script-appropriate font. Unknown/absent locale keeps the configured default
unchanged.
754         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
755         localized_watermark = localize_watermark(stitching.watermark, req.locale)
756         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
757                                 end_padding=stitching.end_padding,
watermark=localized_watermark)
758         try:
759             # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
760             # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
761             # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
762             # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
763             # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
764             local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))

```

16. user (2026-06-22T09:18:21.432Z)

```

455         if cand_after is not None:
456             cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
457             conn.commit()
458
459         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
460             try:
461                 with self._connect() as conn:
462                     cursor = conn.cursor()
463                     cursor.execute(
464                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
465                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
466                     )
467                     conn.commit()
468                     return cursor.lastrowid
469             except Exception as e:
470                 logger.error(f"Failed to add play segment: {e}")
471                 return None
472
473         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:

```

```

474         return self._execute_write(
475             "INSERT INTO events (segment_id, timestamp, type, player, details) VALUES
(?, ?, ?, ?, ?)",
476             (segment_id, timestamp, event_type, player, json.dumps(details or {})),
477             "Failed to add event",
478         )
479

```

17. assistant (2026-06-22T09:19:48.813Z)

I have everything verified. The two count-affecting things confirmed: `min\_shot\_count` compile filter (main.py:735, but `shot\_count` is randomized via `random.randint(4,25)` at motion.py:196 ? so the COMPILED reel count depends on unseeded RNG when `min\_shot\_count > 4`, default is 1 so no segment is filtered) and the transient DB-insert-None path. The `play\_segments` COUNT itself = `len(final\_segments)` from pure-CV detection minus any None-inserts.

Here is the synthesized design.

E2E Reel-Count Nightly Regression ? Design

1) The asserted signal: "number of reels"

****Definition.**** A "reel" is one stitched `.mp4` produced by one `/api/compile` call from N filtered `play\_segments`. The stable, headless, per-video integer to assert is the ****count of candidate highlight clips = `play\_segments` rows for that `video\_id` after segmentation****, with an optional second tier asserting the ****post-`min\_shot\_count` compilable count**** and ****reel-file existence****.

```

signal_primary(video)  = COUNT(*) FROM play_segments WHERE video_id = compute_video_id(path)
signal_compilable(video)= |{ s in play_segments : shot_count is None OR shot_count >=
stitching.min_shot_count }|
signal_reel(video)      = 1 stitched .mp4 per /api/compile call (size > 0)

```

****Headless read (no server, no UI).**** Run the CLI single-shot path (`backend/main.py:997` ? `args.video\_path` and not `args.server` and not `args.app`), which segments and finalizes (destroy-AFTER-confirm at `\_finalize\_reingest`, main.py:1062), then exits. The printed line `Indexed N play segments` (main.py:1060) is N, but the ****authoritative read is the DB**** (immune to log-format drift):

```
indexer --video-path <golden.mp4> --segmenter motion --output-dir output/nightly_e2e
```

then:

```

python -c "from backend.infrastructure.database import Database; \
from backend.utils.hashing import compute_video_id; \
db=Database('output/nightly_e2e/rally_indexer.db'); \
import sqlite3; \
print(db._connect().__enter__().execute('SELECT COUNT(*) FROM play_segments WHERE
video_id=?',(compute_video_id('<golden.mp4>'))).fetchone()[0])"

```

****Verified determinism caveat (load-bearing for the assertion design).****

- `\_detect\_motion\_segments` (motion.py:39-175) is ****pure CV**** (cv2.absdiff/threshold/GaussianBlur/smoothing) ? no RNG, no GPU, deterministic `(start,end)` tuples. So `len(final\_segments)` is deterministic for a fixed (video, `indexing.\*` config, OpenCV build).
- `\_populate\_db\_with\_events` (motion.py:177-252) uses ****unseeded `random`**** (lines 191/196/212/216/218/231/238) ? but ONLY for fabricated metadata/events. The number of `play\_segments` rows = `len(final\_segments)`, NOT affected by RNG. ****`signal\_primary` is RNG-independent.****

- **One RNG leak into the COUNT chain:** ``signal_compilable`` depends on ``metadata.shot_count`` = `random.randint(4,25)` (motion.py:196) vs ``stitching.min_shot_count``. With the **default** ``min_shot_count=1`` (config/models.py:255) NO segment is ever filtered? ``signal_compilable`` == `signal_primary``, deterministic. It becomes RNG-sensitive only if a config raises ``min_shot_count`` above 4. **Design choice:** gate on ``signal_primary``; report ``signal_compilable`` only, and seed ``random`` in the harness to make it deterministic too.
- **Silent-failure leak:** ``add_play_segment`` returns ``None`` on any exception (database.py:469-471); the ``if segment_id:`` guard (motion.py:203) then drops that row. On a transient DB error the count can shrink by 1+ silently. This is rare on a single-process CLI run over a tiny DB, but it argues for a **non-zero retry / fail-loud** posture and for **exact-match** with a **re-run-on-mismatch** rather than blind exact-fail (see §4).

2) Recommended HYBRID design ? two tiers

TIER-A ? local full-pipeline reel-count regression (the faithful test)

- **Where:** the owner's machine (same place PR #202's nightly runs), as a Windows Scheduled Task. NOT in GitHub CI.
- **What it runs:** the **real configured + checkpointed pipeline** end-to-end on **2-3 short owned golden videos** via ``indexer --video-path`` (CLI single-shot). For the production/owned segmenter this uses the **real WASB GPU trajectories** (or their **cached** trajectory CSVs, exactly as PR #202 reuses cached trajectories ? seconds, no GPU re-extraction).
- **Asserts:** ``signal_primary`` per video == frozen baseline (exact, with the re-run guard in §4); reports ``signal_compilable``. Optionally TIER-A+ actually calls the compiler (``VideoCompiler.compile_highlights``, compiler.py:305) on the kept segments and asserts the stitched ``.mp4`` exists and size > 0.
- **CAN test:** the true detector config + real weights/trajectories; the actual DB write path (``add_play_segment``); the config?count contract; (TIER-A+) real ffmpeg stitching + watermark on real frames.
- **CANNOT test:** anything portable/contributor-facing ? it needs the local golden corpus + cached trajectories + GPU/ffmpeg + (optionally) fonts that only exist on the owner box.

TIER-B ? GitHub-CI deterministic CPU-motion smoke (always-on, portable)

- **Where:** ``.github/workflows/ci.yml``, a new fast job (mirrors the existing ``golden-crossos`` job pattern, lines 99-125 ? minimal deps, optional cross-OS matrix).
- **What it runs:** ``indexer --video-path <tiny fixture>.mp4 --segmenter motion`` on a **tiny committed synthetic fixture** (a few seconds, generated by ffmpeg in-job or committed as a small LFS-free ``.mp4``), then asserts ``signal_primary`` == `FROZEN_CI_COUNT``. The motion segmenter imports only `cv2/numpy/scipy/stdlib` (motion.py:1-8) ? **zero torch/CUDA/weights/network**. The ``full-suite`` lane already installs `opencv` system libs (ci.yml:76-79) and CPU torch, so a focused job can ``pip install`` the minimal set.
- **CAN test:** that the segmenter?DB?count contract holds on every PR, portably, deterministically (pure-CV detection is cross-OS stable within tolerance); catches an accidental config/threshold change or a DB-write regression that changes counts.
- **CANNOT test:** the **real WASB model** (the production detector). WASB needs torch + CUDA + model weights + WSL/GPU; those wheels need an out-of-band index (ci.yml:80-84 installs CPU torch only), the weights aren't committed (licensing/size), and GPU runners aren't available in this repo's CI. So CI can only exercise the **motion** segmenter as a stand-in for the pipeline plumbing ? it is a **smoke**, not the faithful detector test. That faithful test is exactly what TIER-A exists for.

Why the split is correct: TIER-A answers "did the **shipped configured+checkpointed** pipeline change how many reels a real golden video yields?" (the owner's question) but can only run where the corpus+weights live. TIER-B answers "did anyone break the segmenter?DB?count plumbing?" portably on every PR. Neither alone suffices; together they cover both the faithful-but-local and the portable-but-shallow ends.

3) Extensible manifest pattern (add a video = one row + freeze its count)

Reuse PR #202's manifest+baseline split. A new committed manifest ``eval_baselines/reelcount_manifest.json`` lists rows; the baseline JSON holds the frozen expected counts (regenerated with ``--update-baseline``).

```
```json
{
```

```

 "_doc": "E2E reel-count manifest. Add a video = append one row; freeze its count via --update-baseline.",
 "segmenter": "motion",
 "config_overrides": {"indexing.motion_threshold": 0.08, "stitching.min_shot_count": 1},
 "videos": [
 {"name": "gx010128_clip", "path": "C:/golden/gx010128_clip.mp4", "trajectory_csv": null},
 {"name": "mp2_clip", "path": "C:/golden/mp2_clip.mp4", "trajectory_csv":
"C:/golden/mp2_clip_wasb.csv"}
]
}
...

```

`trajectory\_csv: null` ? run the live detector; a path ? feed the **cached** WASB trajectory (TIER-A, no GPU). Baseline (regenerated, never hand-edited):

```

```json
{"git_commit":"689c738","generated_at":"2026-06-22","segmenter":"motion",
 "videos":{"gx010128_clip":{"md5":"<video_id>","play_segments":3,"compilable":3},
           "mp2_clip":{"md5":"<video_id>","play_segments":5,"compilable":5}}}
...

```

Adding a video: append one manifest row ? ``python scripts/nightly_e2e_reelcount.py --update-baseline`` ? commit both files. The runner reports added/removed videos as STALE (exit 4) exactly like PR #202's corpus-drift handling (`nightly_regression.py:272-280`), so a new row can't silently pass un-baselined. The CI fixture (TIER-B) is its own tiny manifest row with ``FROZEN_CI_COUNT`` committed.

4) Determinism handling + exact-vs-tolerance

- **signal_primary**: assert EXACT (``==``). Detection is pure-CV deterministic for a fixed (video, config, OpenCV build); the count does not ride on RNG. Tolerance would mask the very regressions we want.
- **Mismatch-handling guard** for the silent-DB-failure edge (`database.py:469`): on a count mismatch, **re-run** that one video once; flag a regression only if the mismatch reproduces. A transient ``add_play_segment`` ? ``None`` shrink won't reproduce, a real config/detector change will. This keeps "exact" honest without flaking on the one known non-determinism source.
- **Seed random** in the harness (``random.seed(0)`` before ``process_video``) so ``signal_compilable`` and any event-dependent secondary assertions are also reproducible ? closes the `motion.py:191-238` RNG, which is otherwise unseeded anywhere in ``backend/``.
- **Cross-OS (TIER-B only)**: pure-CV motion can drift across OpenCV builds at threshold boundaries. Keep the CI fixture **count-robust** (segments well clear of ``motion_threshold`/`min_segment_duration`` boundaries so a sub-epsilon float shift can't flip a segment in/out). If a single-OS guarantee is wanted, pin TIER-B to ``ubuntu-latest`` only; the cross-OS matrix is optional.
- **TIER-A+ stitch byte-output**: do NOT assert reel bytes/duration (ffmpeg/NVENC/watermark-font variance). Assert only **reel count** (``==``), file exists, size > 0 ? counts are stable, bytes are not.

5) Reuse of the PR #202 pattern

Clone the structure of ``scripts/nightly_regression.py`` + ``nightly_regression.ps1`` (already present, verified):

- **Baseline JSON** committed under ``eval_baselines/``, regenerated via ``--update-baseline``, written atomically (``save_baseline``, `nightly_regression.py:532-545`) with ``git_commit`/`generated_at`/`_doc``.
- **Dated report** ``output/nightly_e2e_reelcount/<date>.md`` + ``<date>.json`` (format_report pattern, lines 368-460).
- **Exit codes** mirrored: ``0`` pass, ``1`` reel-count regression, ``2`` operational (manifest/video/ffmpeg missing), ``3`` no baseline, ``4`` stale (corpus/config drift).
- **Pure core / I/O shell split** so the comparison + report formatting are unit-testable with synthetic dicts (no video needed) ? exactly as PR #202 tests ``compare_config`/`summarize_run``.
- **PowerShell wrapper** cloned from ``nightly_regression.ps1``: graceful no-op (exit 0) when checker/corpus absent (lines 52-61), ``LATEST_STATUS.txt`` marker, ``--Pull`` off by default, dated log under ``.../logs/``. Register via the existing ``scripts/register_nightly_task.ps1``.

6) OPEN DECISIONS FOR OWNER

1. **Gate signal ? `play\_segments` count vs compiled-reel count?** Fork: assert the candidate count (deterministic, default `min\_shot\_count=1` makes them equal) vs the post-filter compilable count (RNG-sensitive if floor>4). **Default:** gate `signal\_primary` (play_segments count); report `signal\_compilable`.
2. **TIER-A detector ? live WASB(GPU) vs cached trajectories vs motion-only?** Fork: faithful-but-needs-GPU vs fast-cached (PR #202 style) vs cheapest. **Default:** cached WASB trajectories where available (seconds, faithful to the checkpointed model), live only when re-extracting intentionally.
3. **TIER-A+ actually stitch a reel, or stop at the count?** Fork: count-only (stable, fast) vs count + real ffmpeg stitch (exercises compiler.py:305 + watermark). **Default:** count-only gated; stitch as an optional `--with-stitch` that asserts file-exists/size>0 only.
4. **TIER-B cross-OS ? ubuntu-only vs 3-OS matrix?** Fork: simplest/stable vs broader-but-float-drift-prone. **Default:** ubuntu-only for the gate, optional 3-OS matrix (like `golden-crossos`) with a count-robust fixture.

7) Concrete file / implementation plan

****New files****

- `scripts/nightly\_e2e\_reelcount.py` ? pure core (`summarize\_video`, `compare\_counts`, `format\_report`, exit-code logic) + I/O shell. Runs each manifest video through the CLI segmentation path (in-process: `compute\_video\_id` ? `segmenter.process\_video` with `random.seed(0)` ? `query\_play\_segments` count) or shells `indexer --video-path`. `--update-baseline`, `--with-stitch`, `--manifest`, `--baseline`, `--report-dir` flags. Re-run-once-on-mismatch guard (\$4).
- `scripts/nightly\_e2e\_reelcount.ps1` ? Scheduled-Task wrapper cloned from `nightly\_regression.ps1` (graceful no-op, `LATEST\_STATUS.txt`, dated log).
- `eval\_baselines/reelcount\_manifest.json` + `eval\_baselines/reelcount\_baseline.json` ? manifest (committed) + frozen counts (committed; tied to local corpus, like `nightly\_baseline.json`).
- `tests/test\_nightly\_e2e\_reelcount.py` ? unit tests of the pure core (synthetic count dicts: match?exit 0, drop?exit 1, added video?exit 4, no baseline?exit 3). No video needed; runs in CI's `full-suite`.
- `tests/fixtures/reelcount\_tiny.mp4` (or an ffmpeg-generate step) ? the tiny TIER-B fixture; commit `FROZEN\_CI\_COUNT` in a small `ci\_reelcount\_baseline.json`.

****Edits****

- `.github/workflows/ci.yml` ? add a `reelcount-smoke` job (model on `golden-crossos`, lines 99-125): minimal deps + opencv system libs (lines 76-79), generate/checkout the tiny fixture, run `indexer --video-path` with `--segmenter motion`, assert `signal\_primary == FROZEN\_CI\_COUNT`. ubuntu-only by default.
- `scripts/register\_nightly\_task.ps1` ? register the second nightly task (or extend it to call both wrappers).
- `docs/` ? a tracked project doc per the working-agreement (template \$7 progress tracker + definition-of-done), and a row in `docs/DOC\_STATUS.md`.

****Scheduling (TIER-A):**** Windows Scheduled Task via the new `ps1`, nightly, owner machine (same rationale as `nightly\_regression.ps1` lines 14-16 ? cached trajectories + corpus live outside the repo, so a cloud routine can't run it). ****CI cron (TIER-B):**** the smoke runs on every `pull\_request`/`push` (ci.yml `on:` lines 3-6); add an optional `schedule: cron` trigger if a nightly CI run is also wanted.

****Verification before PR:**** `python run\_all\_tests.py --cov` (green, coverage ? 70 floor, ci.yml:97), `ruff check .`, `mypy backend training`; run TIER-B locally against the tiny fixture; hand TIER-A's real-video/GPU/cached-trajectory verification to the owner (per CLAUDE.md "heaviest machine-side checks").

****Key file:line anchors:**** CLI single-shot `backend/main.py:997`; count finalized after `backend/main.py:1062`; `add\_play\_segment` silent-None
`backend/infrastructure/database.py:469-471`; RNG (count-safe)
`backend/pipeline/segmenters/motion.py:191-238`; pure-CV detection
`backend/pipeline/segmenters/motion.py:39-175`; compile `min\_shot\_count` filter
`backend/main.py:735`; `stitching` defaults `backend/config/models.py:255`; reuse pattern
`scripts/nightly\_regression.py` (save_baseline:532, exit codes:337-342, format_report:368) +

`scripts/nightly\_regression.ps1`; CI portable-job pattern `.github/workflows/ci.yml:99-125`.